Politechnika Świętokrzyska

Aplikacje Internetowe
Podstawy Technologii Internetowych

Dr inż. **Wikarek Jarosław**
Katedra Informatyki Stosowanej
e-mail: j.wikarek@tu.kielce.pl
pokój: d408

Wydział Elektrotechniki, Automatyki i Informatyki

JavaScript

Język programowania, służący do tworzenia interaktywnych i dynamicznych funkcji na stronach internetowych. Nadaje stronom ruch, reaguje na kliknięcia użytkownika oraz pozwala aktualizować dane bez przeładowywania całej witryny

JavaScript

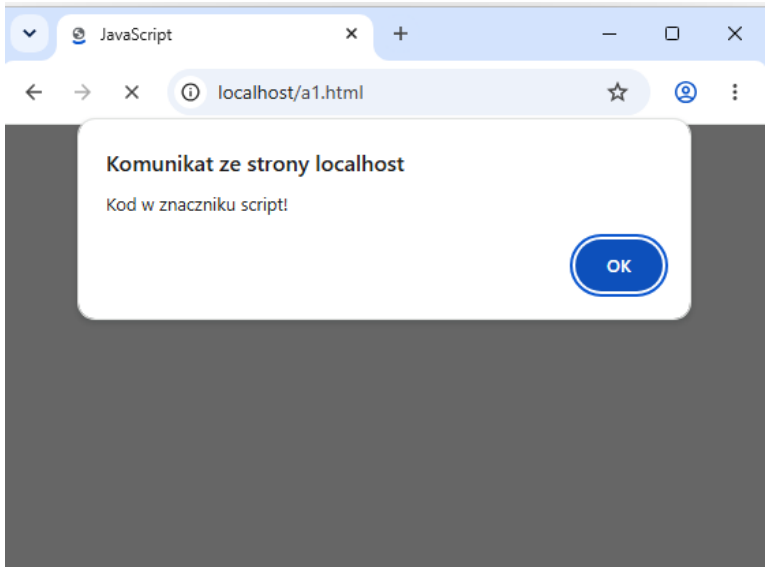
- JavaScript to skryptowy język programowania, stworzony przez firmę Netscape.
- Najczęściej spotykanym zastosowaniem języka JavaScript są strony internetowe.
- Skrypty te służą do zapewnienia interakcji poprzez:
  - ✓ reagowanie na zdarzenia,
  - ✓ walidacji danych wprowadzanych w formularzach,
  - ✓ tworzenia złożonych efektów wizualnych.
- Skrypty JavaScriptu uruchamiane przez strony internetowe mają znacznie ograniczony dostęp do komputera użytkownika. Po stronie serwera JavaScript może działać w postaci np. node.js czy Ringo.

Wysyłanie wyników do przeglądarki

- Metoda `alert()`

a1.html

```
1  <html>
2   <head>
3      <title>JavaScript</title>
4   </head>
5   <body>
6    <script>
7      alert("Kod w znaczniku script!");
8    </script>
9   </body>
10 </html>
11
```

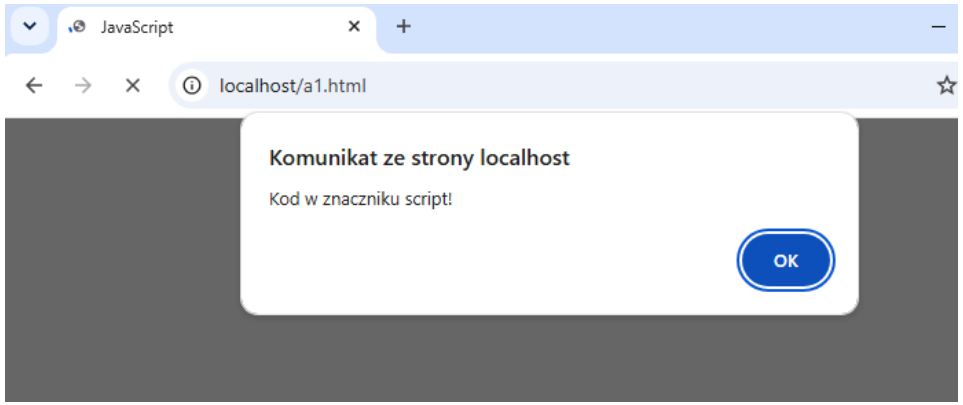


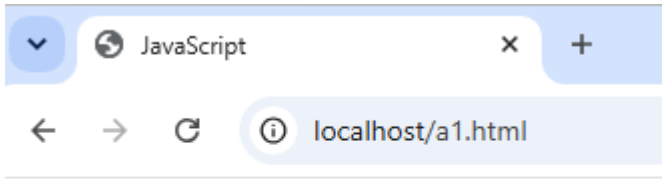
Załączanie skryptu

- Lokalnie w pliku `html` umieszczając kod pomiędzy znacznikami

a1.html

```
1  <html>
2   <head>
3      <title>JavaScript</title>
4   </head>
5   <body>
6    <script>
7      alert("Kod w znaczniku script!");
8    </script>
9   </body>
10 </html>
11
```



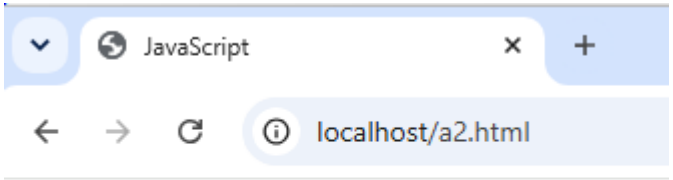


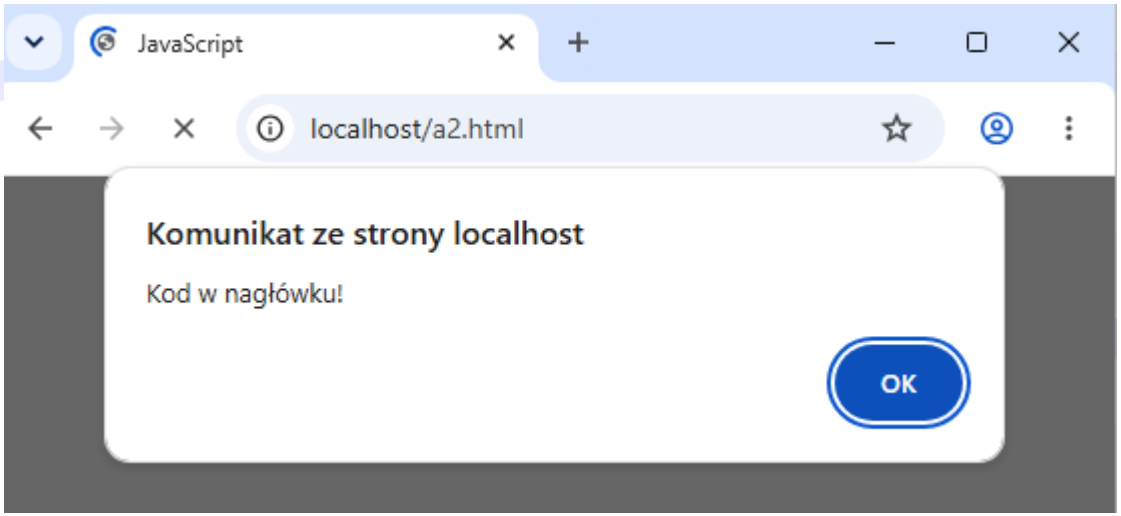
Załączanie skryptu

- Lokalnie w pliku `html` umieszczając kod w nagłówku

a2.html

```
1  <html>
2   <head>
3      <title>JavaScript</title>
4    <script>
5       alert("Kod w nagłówku!");
6    </script>
7   </head>
8   <body>
9
10  </body>
11 </html>
12
```





Załączanie skryptu

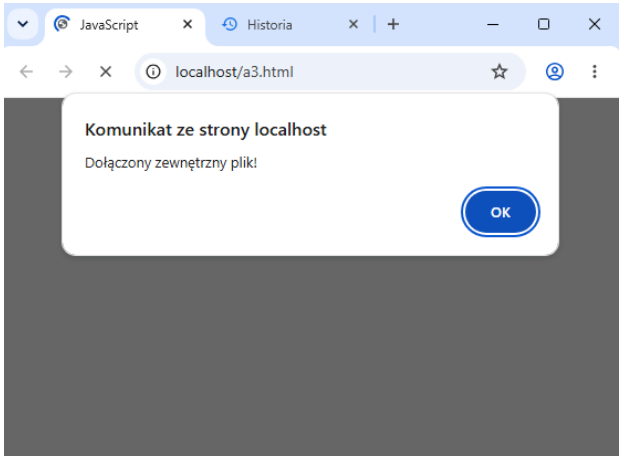
- Skrypt osadzony w zewnętrznym pliku – **zalecany sposób**

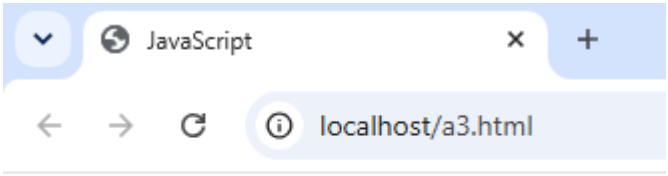
a3.html | plik1.js

```
1  <html>
2   <head>
3     <title>JavaScript</title>
4     <script src="plik1.js"> </script>
5   </head>
6   <body>
7    <script>
8      powitanie();
9    </script>
10  </body>
11 </html>
12
```

a3.html

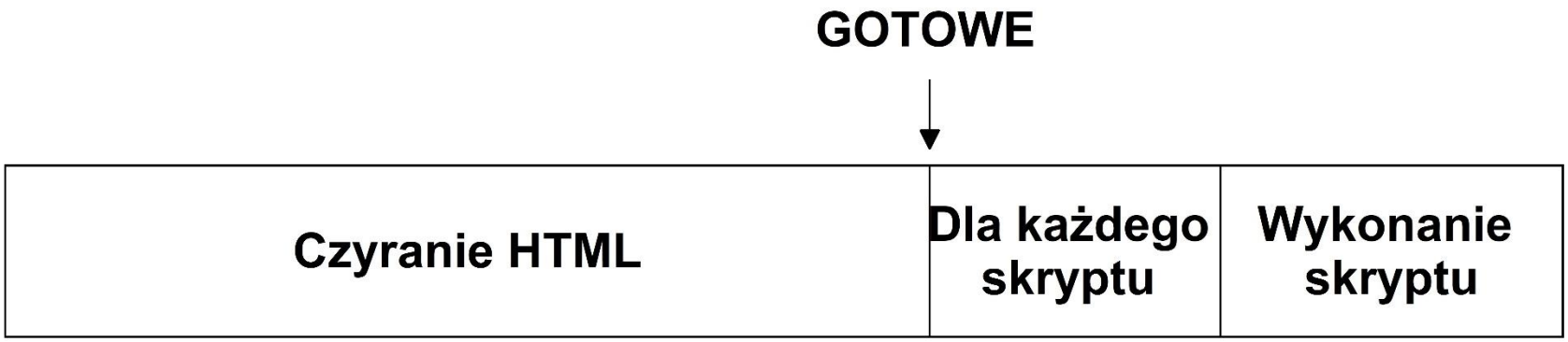
```
function powitanie()
{
    alert("Dołączony zewnętrzny plik!");
};
```





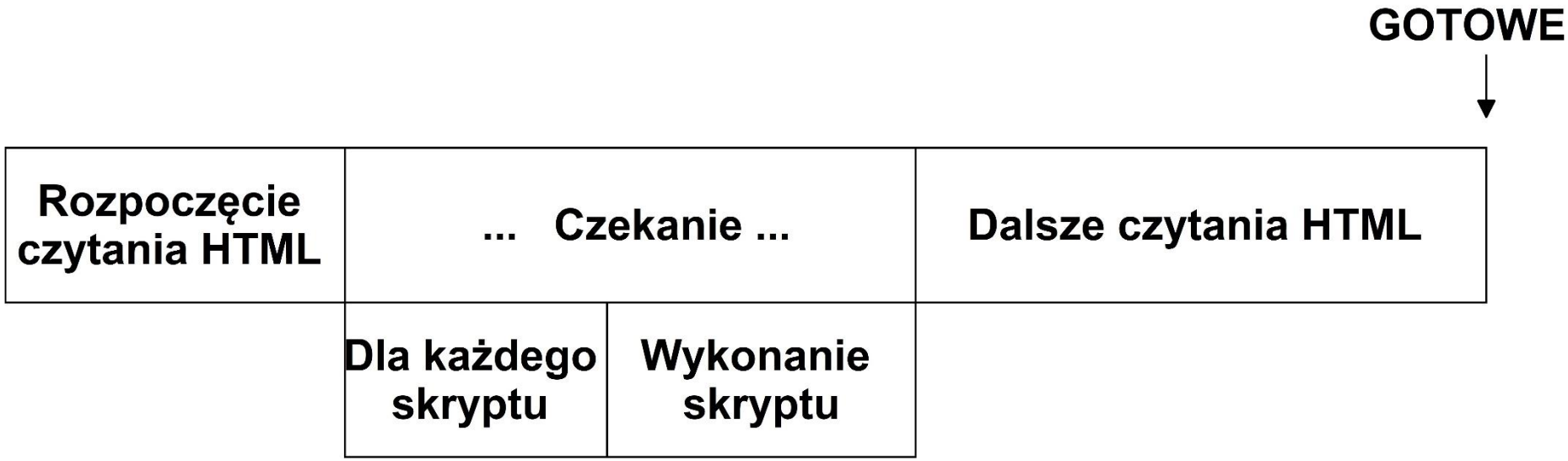
Kiedy skrypt się ładuje

- Skrypt osadzony w zewnętrznym pliku
  - ✓ Skrypt wczytany na koniec dokumentu html w sekcji <body>



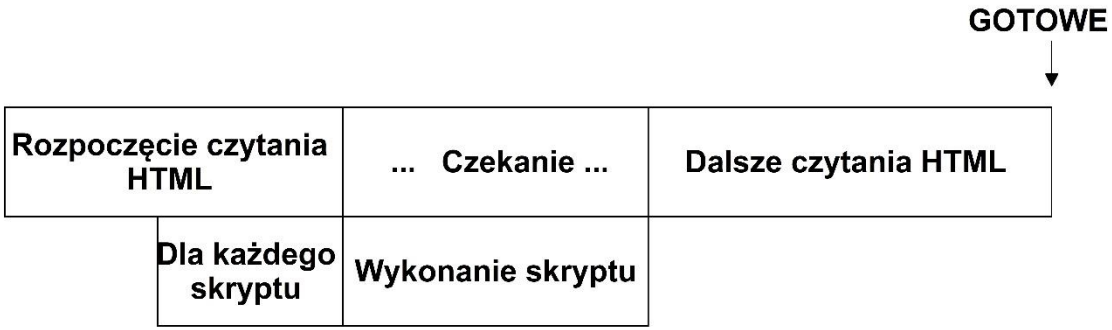
Kiedy skrypt się ładuje

- Skrypt osadzony w zewnętrznym pliku
  - ✓ Skrypt wczytany bezpośrednio w sekcji <head>

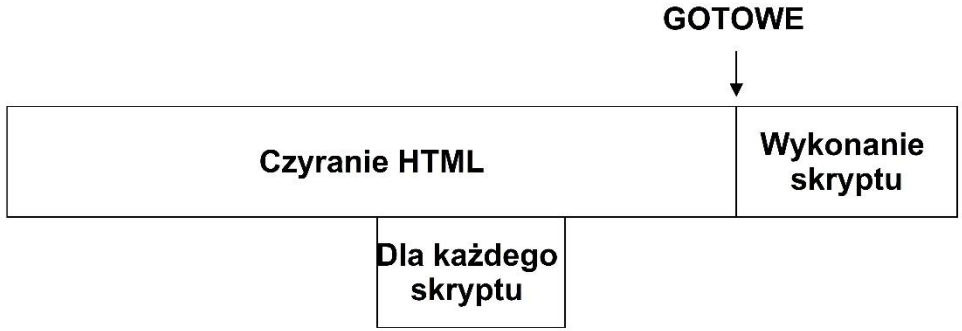


Kiedy skrypt się ładuje

- ✓ Async pobiera skrypt w tle i wykonuje go natychmiast po pobraniu (przerywając parsowanie HTML



- ✓ defer pobiera skrypt w tle, ale czeka z jego wykonaniem do końca parsowania



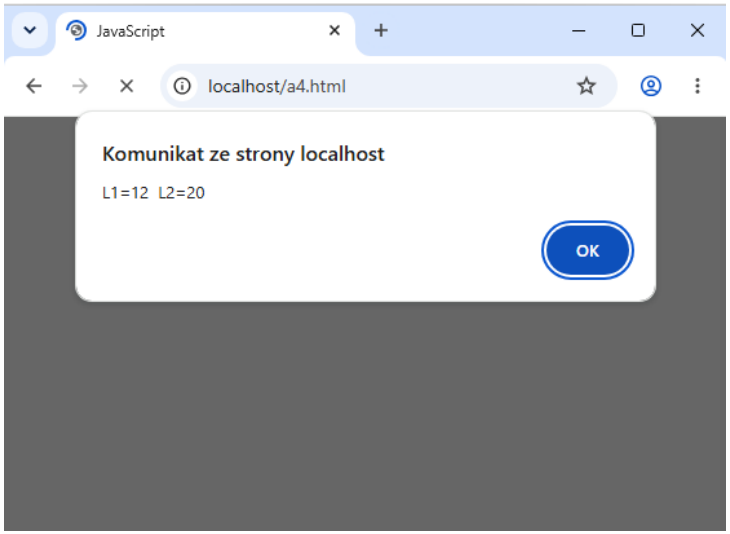
Istotna różnica między tymi atrybutami polega na tym, że skrypty z atrybutem **defer** będą uruchamiane w kolejności, w jakiej zostały umieszczone w dokumencie, tymczasem skrypty z atrybutem **async** podlegają kolejności wczytywania się – jako pierwszy uruchomi się ten skrypt, który zostanie wczytany jako pierwszy.

Zmienne

- Tworząc nazwę zmiennej, nie możemy: rozpocząć jej od cyfry; użyć spacji w nazwie; wykorzystywać słów kluczowych języka JavaScrip; unikać polskich znaków diakrytycznych (choć nie jest to zakazane)

a4.html

```
1  <html>
2   <head>
3     <title>JavaScript</title>
4    </head>
5   <body>
6    <script>
7      var l1 = 12;
8      let l2 = 20;
9      alert("L1="+l1+"  L2="+l2);
10    </script>
11   </script>
12  </body>
13 </html>
```



let – ma zasięg lokalny; var – globalny

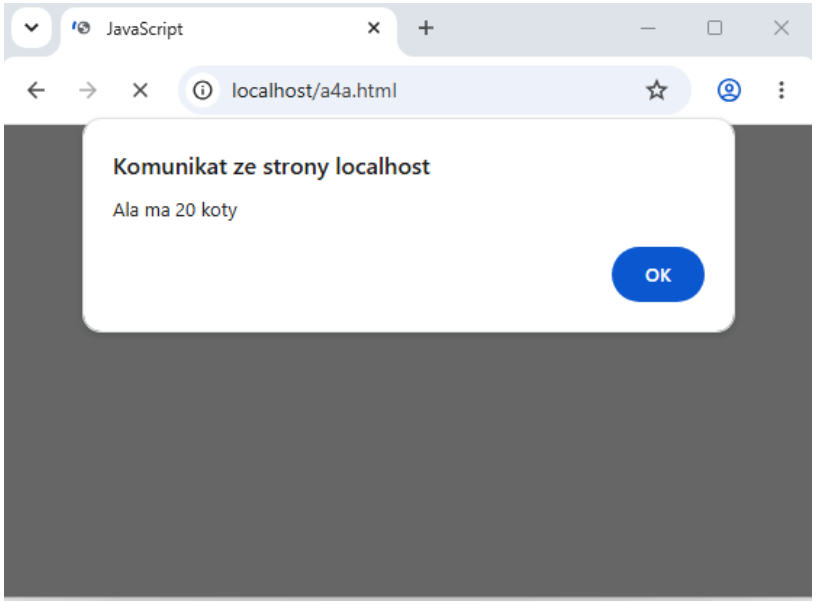
Stałe

- Oprócz zmiennych, w których przechowywane wartości mogą się dynamicznie zmieniać w trakcie wykonania skryptu, w języku JavaScript możemy też zadeklarować tzw. stałą. Jest to taki pojemnik na dane, którego zawartość nie ulega zmianom.

a4a.html

C:\Oprogramowanie\Apache24\htdocs\a4a.html

```
2   <head>
3     <title>JavaScript</title>
4    </head>
5   <body>
6     <script>
7       const c1 = "Ala ma ";
8       const c2 = 20;
9       const c3 = " koty";
10      alert(c1+c2+c3);
11    </script>
12   </script>
13  </body>
14 </html>
15
```

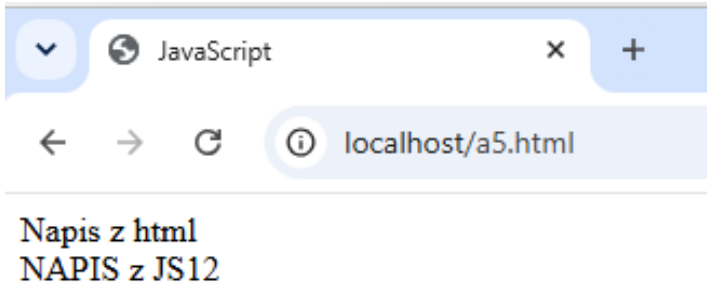


Wysyłanie wyników do przeglądarki

- Metoda `document.write()`

a5.html

```
1  <html>
2   <head>
3     <title>JavaScript</title>
4    </head>
5   <body>
6    Napis z html <BR>
7    <script>
8      var l1 = 12;
9      document.write("NAPIS z JS  "+l1);
10    </script>
11   </script>
12  </body>
13 </html>
14
```

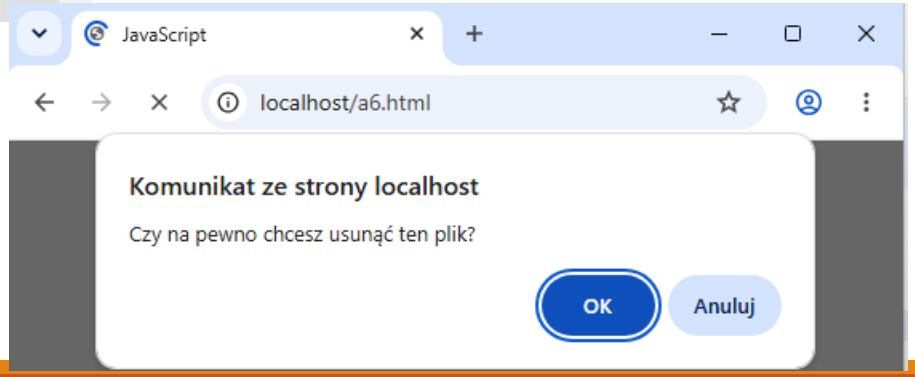


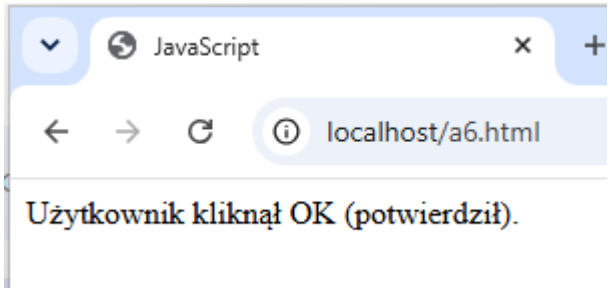
Wysyłanie wyników do przeglądarki

- Metoda `window.confirm()`

a6.html

```
1  <html>
2   <head>
3     <title>JavaScript</title>
4    </head>
5   <body>
6    <script>
7     let wynik = window.confirm("Czy na pewno chcesz usunąć ten plik?");
8     if (wynik) {document.write("Użytkownik kliknął OK (potwierdził).");}
9     else {document.write("Użytkownik kliknął Anuluj (odrzucił).");}
10   </script>
11  </body>
12 </html>
```



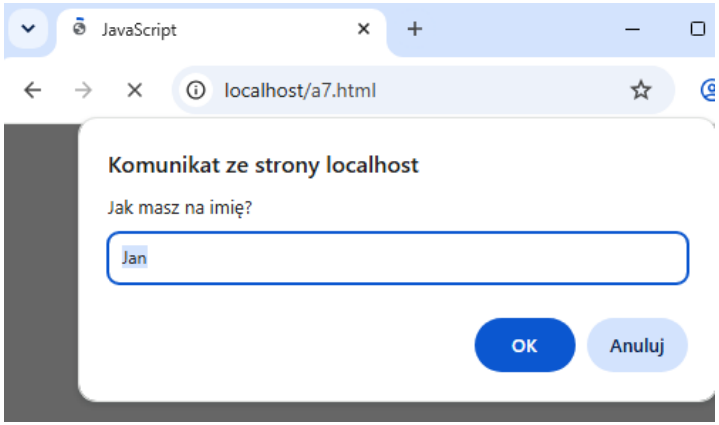


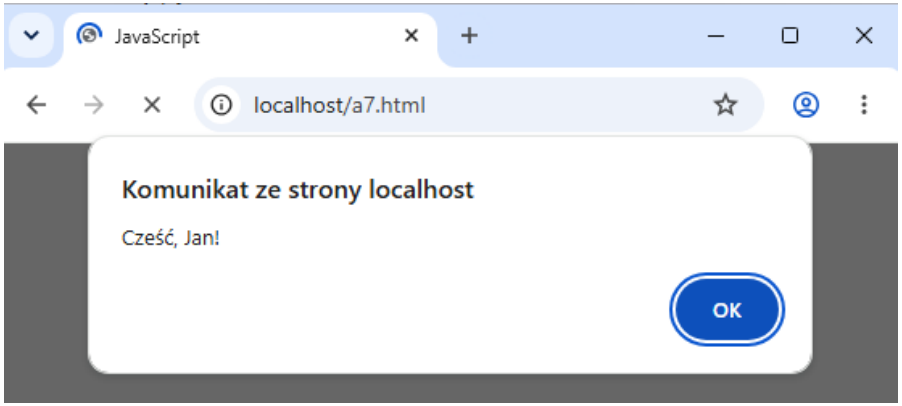
Wysyłanie wyników do przeglądarki

- Metoda `window.prompt()`

a7.html

```
1  <html>
2   <head>
3     <title>JavaScript</title>
4    </head>
5   <body>
6    <script>
7     let imie = window.prompt("Jak masz na imię?", "Jan");
8     if (imie !== null && imie.trim() !== "") { alert("Cześć, " + imie + "!");}
9     else {alert("Anulowano lub nic nie wpisano.");}
10   </script>
11  </body>
12 </html>
```





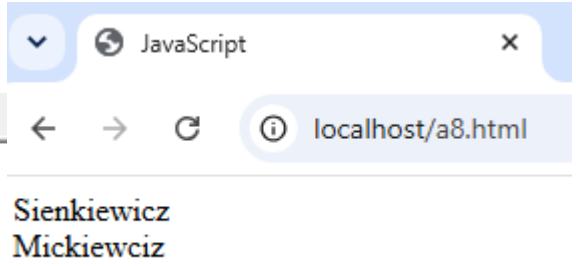
Tablice

- Tablica w JavaScript to specjalny obiekt służący do przechowywania wielu wartości w jednej zmiennej. Elementy w tablicy mają swoje numery, zwane indeksami, które zawsze zaczynają się od zera. Tablice tworzy się za pomocą nawiasów kwadratowych

a8.html

C:\Oprogramowanie\Apache24\htdocs\a8.html

```
1  <html>
2   <head>
3     <title>JavaScript</title>
4    </head>
5   <body>
6    <script>
7     var   ksiazka1 = {autor : "Sienkiewicz", tytul : "Potop"      , oprawa : "Twarda"};
8     var   ksiazka2 = {autor : "Mickiewciz",  tytul : "Pan Tadeusz", oprawa : "Mieka"};
9     let ksiazki = [ksiazka1,ksiazka2];
10    document.write(ksiazki[0].autor);
11    document.write("<BR/>");
12    document.write(ksiazki[1].autor);
13   </script>
14  </body>
15 </html>
```

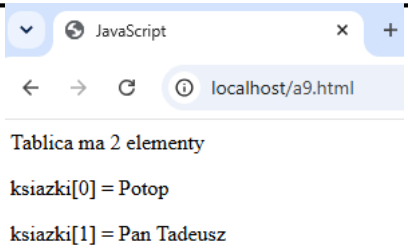


JavaScript

Tablice – przydatne metody

- Liczba elementów tablicy `table.length`
- Poszczególne elementy tablicy

```
table.forEach(function(item, index)
```



a9.html

```
1  <html>
2   <head>
3     <title>JavaScript</title>
4    </head>
5   <body>
6    <script>
7     var   ksiazka1 = {autor : "Sienkiewicz", tytul : "Potop"      , oprawa : "Twarda"};
8     var   ksiazka2 = {autor : "Mickiewciz",  tytul : "Pan Tadeusz", oprawa : "Mieka"};
9     let ksiazki = [ksiazka1,ksiazka2];
10    document.write("Tablica ma "+ksiazki.length+" elementy");
11    document.write("<BR/>");
12    ksiazki.forEach(function(item, index)
13    {
14     document.write("<p>ksiazki[" + index + "] = " + item.tytul + "</p>");
15    });
16   </script>
17  </body>
18 </html>
```

Tablice – przydatne metody

- Metoda `splice()` służy do zmieniania zawartości tablicy. Możesz nią usuwać, dodawać lub zamieniać elementy. Uwaga: ta metoda zmienia oryginalną tablicę i zwraca nową tablicę z usuniętymi elementami.

```
tablica.splice(start,ileUsunąć,element1,...)
```

start indeks, od którego zaczynamy zmiany zmiany.

ileUsunąćl iczba elementów do usunięcia.

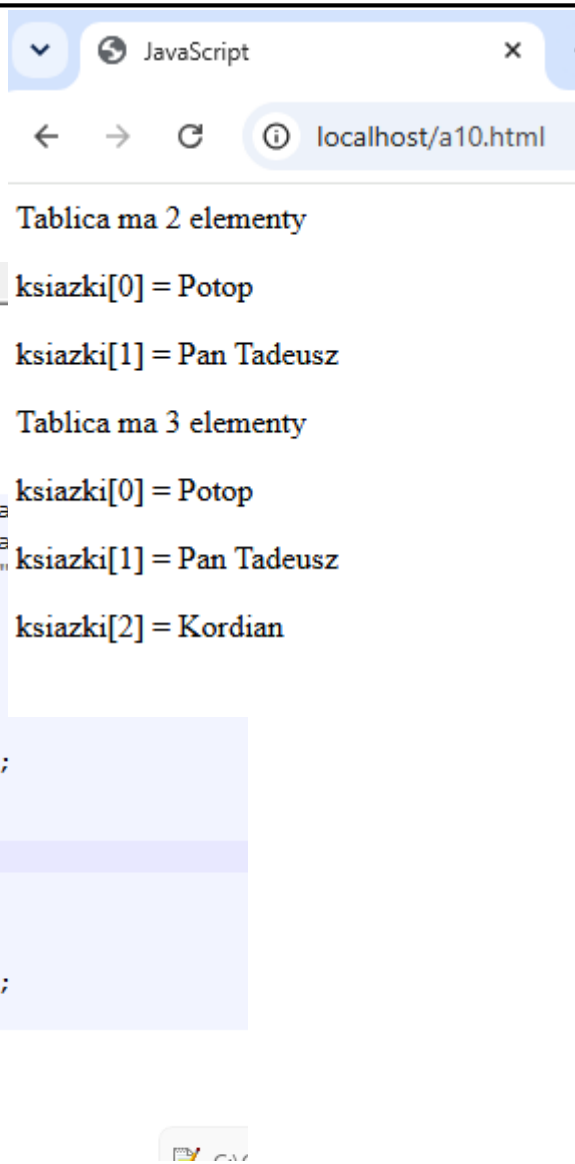
element1 nowe elementy do wstawienia (opcjonalnie).

Tablice – przydatne metody

- Dodanie nowej książki

a10.html

```
1  <html>
2   <head>
3     <title>JavaScript</title>
4    </head>
5   <body>
6    <script>
7     var   ksiazka1 = {autor : "Sienkiewicz", tytul : "Potop"       , oprawa
8     var   ksiazka2 = {autor : "Mickiewciz",  tytul : "Pan Tadeusz", oprawa
9     var   ksiazka3 = {autor : "Słowacki",    tytul : "Kordian", oprawa : "
10    let ksiazki = [ksiazka1,ksiazka2];
11    document.write("Tablica ma "+ksiazki.length+" elementy");
12    document.write("<BR/>");
13    ksiazki.forEach(function(item, index)
14    {
15     document.write("<p>ksiazki[" + index + "] = " + item.tytul + "</p>");
16    });
17    ksiazki.splice(3,0,ksiazka3);
18    document.write("Tablica ma "+ksiazki.length+" elementy");
19    document.write("<BR/>");
20    ksiazki.forEach(function(item, index)
21    {
22     document.write("<p>ksiazki[" + index + "] = " + item.tytul + "</p>");
23    });
24   </script>
25  </body>
26 </html>
27
```



Tablice – inne przydatne metody

- **`includes()`** Sprawdza, czy element istnieje w tablicy. Zwraca true lub false.
- **`indexOf()`** Zwraca indeks (pozycję) pierwszego pasującego elementu. Zwraca -1, gdy go nie ma.
- **`find()`** Zwraca wartość pierwszego elementu, który spełnia warunek w funkcji. Zwraca undefined, gdy nic nie znajdzie.
- **`findIndex()`** Zwraca indeks pierwszego elementu spełniającego warunek. Zwraca -1, gdy go nie ma.
- **`some()`** Sprawdza, czy choć jeden element spełnia warunek. Zwraca true lub false.

Tablice – inne przydatne metody

- **`includes()`** Sprawdza, czy element istnieje w tablicy. Zwraca true lub false.
- **`indexOf()`** Zwraca indeks (pozycję) pierwszego pasującego elementu. Zwraca -1, gdy go nie ma.
- **`find()`** Zwraca wartość pierwszego elementu, który spełnia warunek w funkcji. Zwraca undefined, gdy nic nie znajdzie.
- **`findIndex()`** Zwraca indeks pierwszego elementu spełniającego warunek. Zwraca -1, gdy go nie ma.
- **`some()`** Sprawdza, czy choć jeden element spełnia warunek. Zwraca true lub false.

`https://kursjs.pl/kurs/super-podstawy/tablice`

Reagowanie na zdarzenia

- Typy zdarzeń – związane z formularzem

Nazwa zdarzenia	Zastosowanie, czyli kiedy wystąpienie zdarzenia zostanie zarejestrowane podczas nasłuchiwania
`click`	Klasyczne kliknięcie, które rozumiemy jako naciśnięcie przycisku myszy, a następnie jego zwolnienie, które nastąpiło w czasie, gdy wskaźnik (kursor) myszy wciąż znajdował się na elemencie (obiekcie).
`dblclick`	Dwukrotne kliknięcie elementu (z ang. double click).
`mousedown`	Wystąpi w sytuacji, gdy przycisk myszy został wciśnięty, kiedy kursor znajdował się na elemencie. W odróżnieniu od zdarzenia `click` kursor nie musi znajdować się na obiekcie w momencie zwalniania przycisku.
`mouseup`	Wystąpi w sytuacji, gdy przycisk myszy został zwolniony, kiedy kursor znajdował się na elemencie. W odróżnieniu od zdarzenia `click` kursor nie musi znajdować się na obiekcie w momencie, gdy przycisk myszy został wciśnięty.
`mousemove`	Rejestrowane za każdym razem, gdy kursor myszy poruszy się, znajdując się na elemencie.
`mouseenter`	Zachodzi, kiedy wskaźnik myszy wkroczy z obszaru spoza elementu i znajdzie się na obiekcie. Jednak w odróżnieniu od zdarzenia `mousemove` nie zostanie zarejestrowane ponownie, dopóki kursor nie opuści obiektu.
`mouseover`	Zachodzi, kiedy wskaźnik myszy wkroczy z obszaru spoza elementu i znajdzie się na obiekcie lub także na jego elementach potomnych w hierarchii DOM.
`mouseleave`	Zostanie zarejestrowane, gdy kursor myszy opuści dany obiekt.

Reagowanie na zdarzenia

- Typy zdarzeń – związane z myszą

Nazwa zdarzenia	Zastosowanie, czyli kiedy wystąpienie zdarzenia zostanie zarejestrowane podczas nasłuchiwania
`submit`	Wystąpi w momencie podsumowania (wysłania) danych całego formularza.
`reset`	Zachodzi w sytuacji zresetowania stanu formularza, czyli wprowadzonych do jego kontrolek danych wejściowych (tekstu, wyborów, wskazań).
`change`	Zachodzi w momencie, gdy zmienia się zawartość lub stan kontrolki formularza – np. zmieniono tekst w polu `<input>` lub `<textarea>` albo wskazano opcję na liście wyboru `<select>`.
`select`	Rejestrowane w sytuacji, gdy użytkownik zaznaczy tekst w polu `<input>` lub `<textarea>`.
`input`	Zostanie zarejestrowane, gdy w polu `<input>` formularza pojawią się dane wejściowe, czyli w praktyce wpisany przez użytkownika tekst.

Pola formularza a zdarzenia

- Typy zdarzeń – interfejs witryny

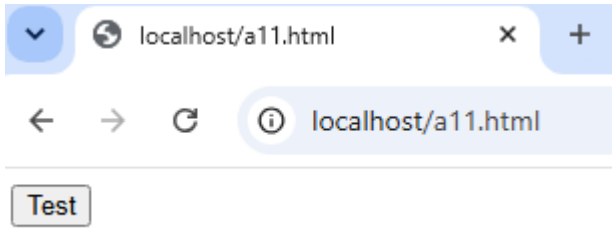
Nazwa zdarzenia	Zastosowanie, czyli kiedy wystąpienie zdarzenia zostanie zarejestrowane podczas nasłuchiwania
`focus`	Wystąpi w momencie, gdy element HTML znajdzie się w centrum uwagi – np. gdy używamy w witrynie klawisza `Tab`, kolejne elementy będą właśnie uzyskiwać wyróżnienie (ostrość, uwagę) w interfejsie.
`blur`	W przeciwieństwie do zdarzenia `focus`, zostanie zarejestrowane w momencie, gdy element przestanie być w centrum uwagi, czyli utraci wyróżnienie (ostrość, uwagę) w interfejsie.
`resize`	Zachodzi w momencie zmiany rozmiaru widoku dokumentu w przeglądarce.
`load`	Rejestrowane, kiedy przeglądarka zakończy wczytywanie podstrony.
`unload`	Zachodzi jednokrotnie w momencie, gdy użytkownik opuszcza interfejs – np. kiedy karta przeglądarki zostaje zamknięta lub gdy naciśnięto przycisk `Wstecz`.
`drag`	Występuje w sytuacji, gdy dany obiekt jest przenoszony w interfejsie.

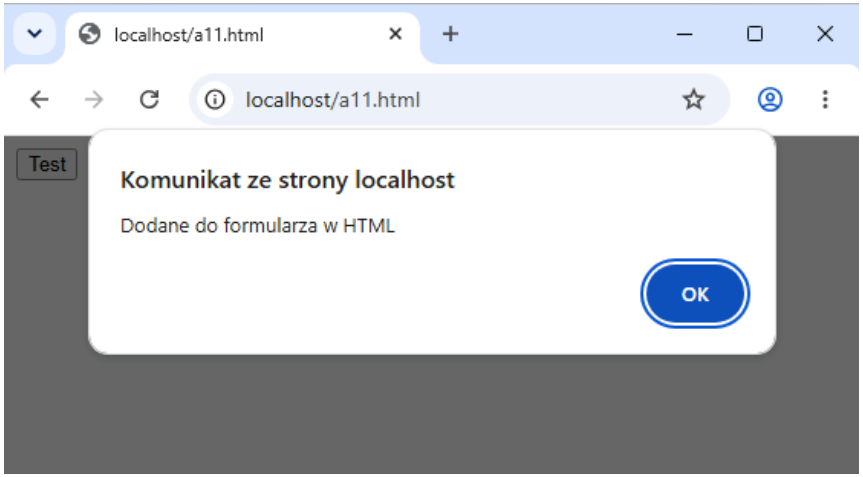
Reagowanie na zdarzenia

- Pola w HTML

a11.html

```
1
2   <input type="button" id="test" value="Test" onClick=hello()>
3   <script>
4     function hello()
5     {
6     alert("Dodane do formularza w HTML");
7     }
8   </script>
```



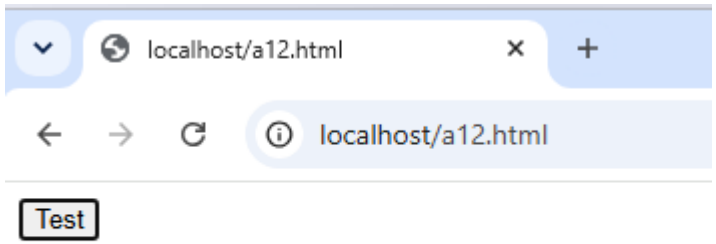


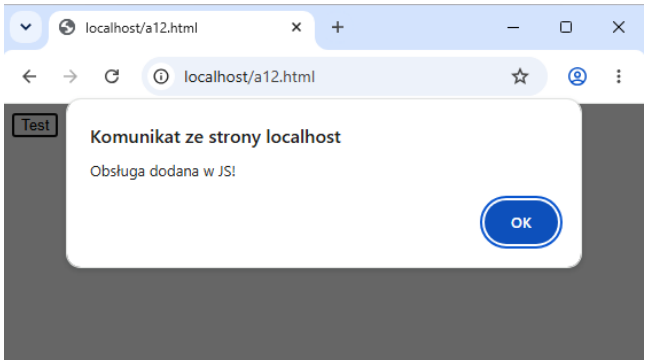
Reagowanie na zdarzenia

- Dodane w JS (metoda **getElementByID** oraz **addEventListner**)

a12.html

```
1
2   <input type="button" id="test" value="Test">
3   <script>
4     function hello()
5   {
6   alert("Obsługa dodana w JS!");
7   }
8   var przycisk = document.getElementById("test");
9   przycisk.addEventListener("click", hello);
10  </script>
```





Reagowanie na zdarzenia

- Aby zareagować na wykonanie zdarzenia, należy podpiąć do danego elementu funkcję nasłuchującą, która zostanie uruchomiona w momencie wykonania danego zdarzenia.

```
element.addEventListener()
```

- Przyjmuje ona 3 wartości:
  - ✓ typ zdarzenia,
  - ✓ funkcję, która zostanie odpalona w momencie zaistnienia zdarzenia.

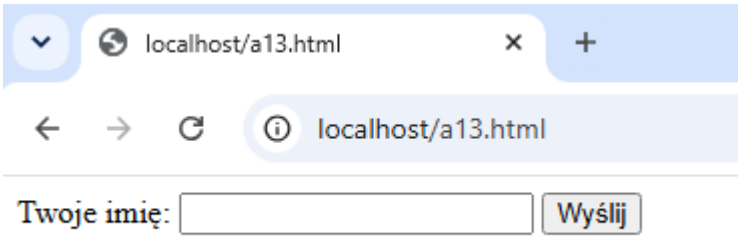
Reagowanie na zdarzenia

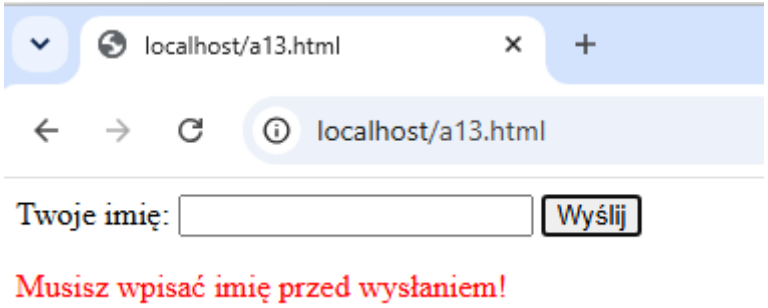
- Podpinając daną funkcję nasłuchującą, możemy ustawić jej parametr do którego Javascript każdorazowo przekaże nam dodatkowy obiekt z informacjami związanymi z tym zdarzeniem. Najczęściej używa się dla tego obiektu metody **preventDefault()** która pozwala anulować domyślne działania przeglądarki. Na przykład, sprawić, aby po kliknięciu linku nie nastąpiło przejście na inną stronę lub po naciśnięciu przycisku formularz nie został wysłany na serwer.

Przykład sprawdzenie czy wypełniono pole formularza

a13.html

```
1  <form id="F1">
2    <label for="name">Twoje imię:</label>
3    <input type="text" id="imie" name="name">
4    <button type="submit">Wyślij</button>
5    <p id="blad" style="color: red;"></p>
6  </form>
7
8  <script>
9    const formularz = document.getElementById('F1');
10   const imie_w    = document.getElementById('imie');
11   const komunikat = document.getElementById('blad');
12   function obsluga(event) {
13   if (imie_w.value.trim() === '')
14     {
15      event.preventDefault();
16      komunikat.textContent = 'Musisz wpisać imię przed wysłaniem!';
17     }
18   }
19   formularz.addEventListener('submit', obsluga);
20 </script>
```

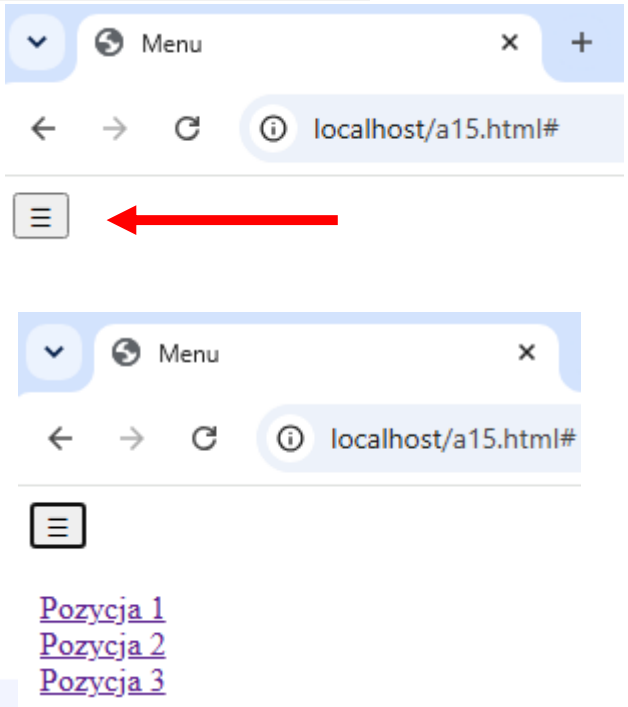




Inne przykłady – rozwijane menu

a15.html

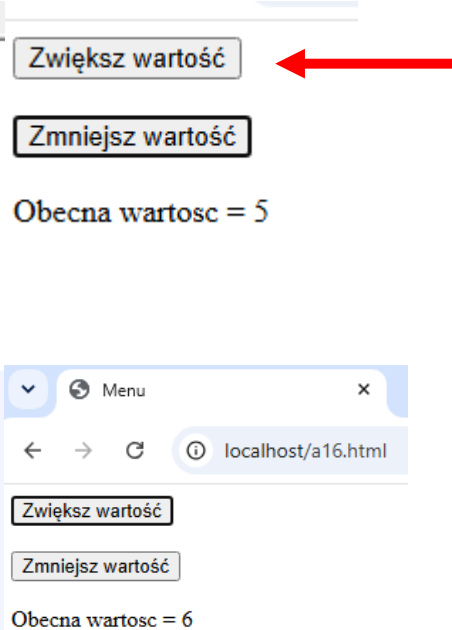
```
1  <html>
2   <head>
3    <title>Menu</title>
4    <style type="text/css">
5      .hidden { display: none; }
6      ul { list-style: none; padding: 5;}
7    </style>
8   </head>
9   <body>
10   <nav>
11     <button id="m1" aria-label="Otwórz menu">☰</button>
12     <ul id="l1" class="hidden">
13      <li><a href="#">Pozycja 1</a></li>
14      <li><a href="#">Pozycja 2</a></li>
15      <li><a href="#">Pozycja 3</a></li>
16     </ul>
17   </nav>
18   <script>
19    const p1   = document.getElementById('m1');
20    const menu = document.getElementById('l1');
21    p1.addEventListener('click', () => {menu.classList.toggle('hidden');});
22    </script>
23  </body>
24 </html>
```



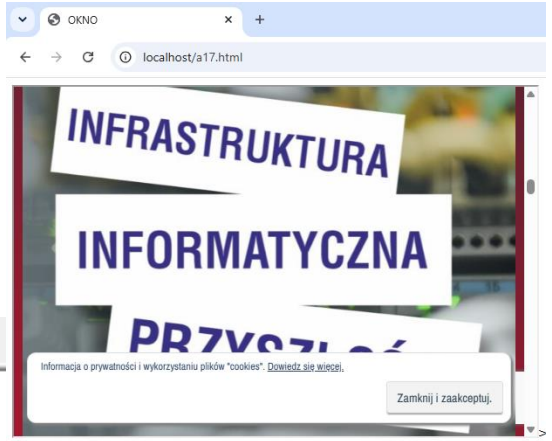
Inne przykłady – interakcja miedzy przyciskami

a16.html

```
1  <html>
2   <head>
3    <title>Menu</title>
4   </head>
5   <body onload=wp()>
6      <button id="p1" onclick=dodaj()>Zwiększ wartość </button> <BR/><BR/>
7      <button id="p2" onclick=odejmij()>Zmniejsz wartość</button> <BR/><BR/>
8      <div id="wartosc" ></div>
9
10   <script>
11    var war=5;
12    function wp()
13    {
14      document.getElementById("wartosc").innerText = "Obecna wartosc = "+war;
15    };
16    function dodaj()
17    { war = war + 1;
18      document.getElementById("wartosc").innerText = "Obecna wartosc = "+war;
19    };
20     function odejmij()
21    { war = war - 1;
22      document.getElementById("wartosc").innerText = "Obecna wartosc = "+war;
23    };
24    </script>
25  </body>
26 </html>
27
```



Metody przydatne do wykonania laboratorium



a17.html | menu.js | menu.css | m_1_1.html | menu.html

```
1  <html>
2  <script>
3    function fr()
4    {
5      document.getElementById("okno_f").src = "http://www.tu.kielce.pl";
6    };
7  </script>
8
9    <head>
10    <title>OKNO</title>
11   </head>
12   <body onload=fr()>
13      <div id='okno'> <iframe id="okno_f" width="600" height="400"></iframe> </div>
14   </body>
15 </html>
16
```

Metody przydatne do wykonania laboratorium

- `date()` konstruktor nowego obiektu zawracający informacje o czasie na serwerze

```
const teraz = new Date();
```

(ustawienie polskiego formatu daty)

```
teraz.toLocaleDateString('pl-PL')
```

```
let godziny = teraz.getHours();
let minuty =  teraz.getMinutes();
let sekundy = teraz.getSeconds()
```

Metody przydatne do wykonania laboratorium

- Instrukcja warunkowa `if`

```
if warunek {kod}
   else if (drugi Warunek) { kod }
   else { kod }
```

- Inny układ if

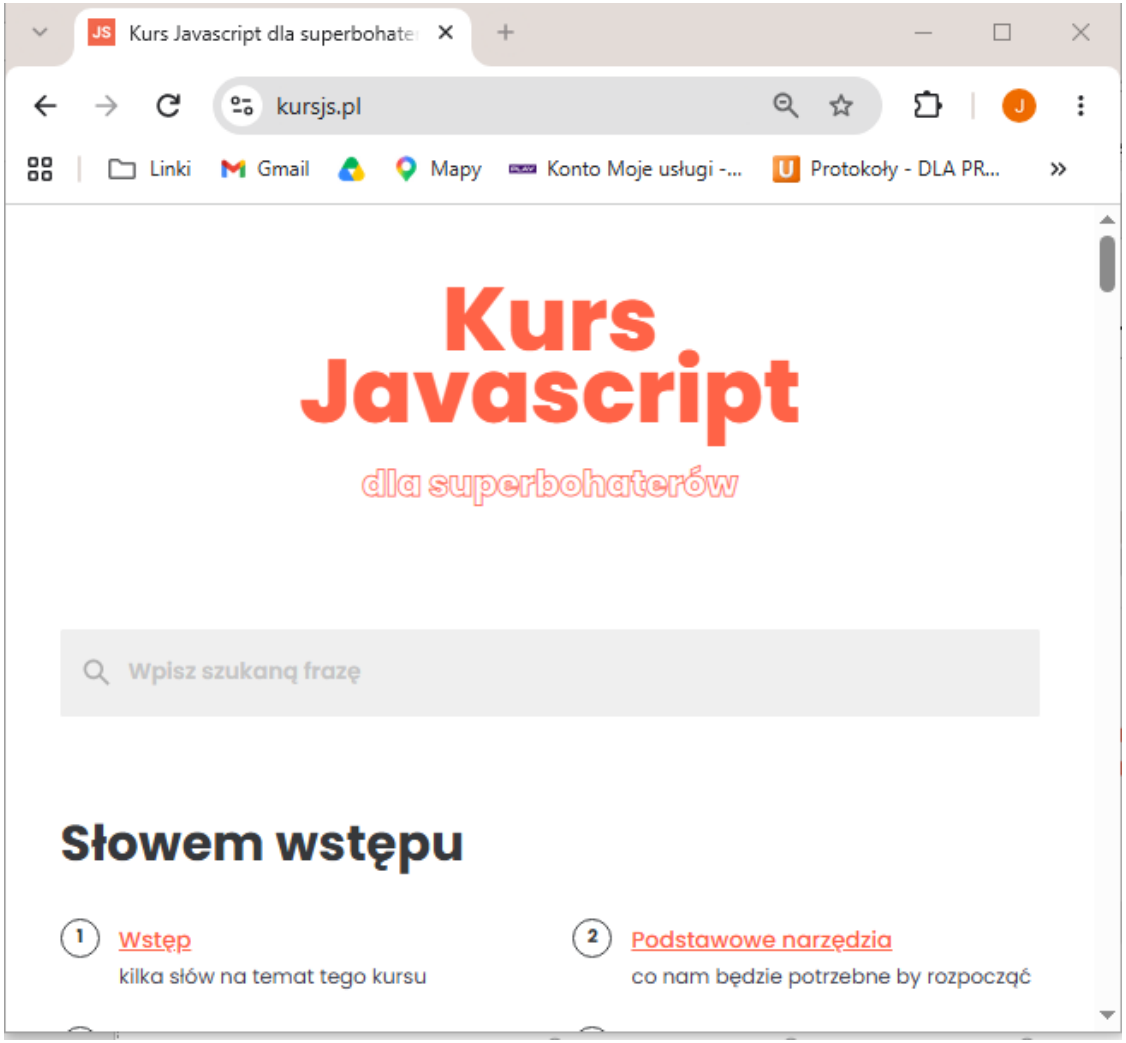
```
warunek ? {kod} : {kod}
```

- Uruchomienie funkcji po określonym czasie (w milisekundach)

```
setTimeout(funkcja, 1000);
function co_s()
{
  setTimeout(co_s, 1000);
} ;
```

Przykładowe kursy

- https://kursjs.pl/



Przykładowe kursy

- https://javascript.info/

